Description Submission view

Available from: Cuma, 13 Şubat 2026, 11:15 AM
Due date: Cuma, 13 Şubat 2026, 12:40 PM
Requested files: SilverSpiral.java (Download)
Type of work: Individual work
Reduction by automatic evaluation: 10 **Free evaluations**: 2**Reduction by automatic evaluation**: 10 **Free evaluations**: 2

CSE 102L Lab 1: The Silver Spiral

Overview

In this laboratory assignment, you will continue your practice with 2D arrays and algorithmic logic. While similar to the Golden Spiral, this task introduces the **Lucas Sequence** and requires you to fill the matrix in a **counter-clockwise spiral pattern**.
The Lucas sequence is a cousin of the Fibonacci sequence but starts with different initial values, leading to a "silver" ratio approximation.

Objectives

1. Implement input validation using the Scanner class.
2. Generate Lucas numbers iteratively.
3. Manipulate 2D array indices to create a **counter-clockwise** spiral.
4. Perform calculations on specific array elements (diagonals).

Method Specifications

You are required to implement the following methods. Do not change the method names, parameters, or return types.

1. getMatrixSize

This method handles user interaction to determine the size of the matrix.

- **Purpose:** Prompt the user to enter an integer N representing the dimensions of the square matrix ($N \times N$).
- **Validation:** The input must be checked to ensure it falls within the range **1 to 6** (inclusive).
- **Error Handling:** If the user enters a number outside this range, the program must print an error message exactly as "ERROR: N must be between 1 and 6." and immediately terminate the program using the system exit command.
- **Return Value:** A valid integer N.

2. getLucas

This method generates a specific number from the Lucas sequence.

- **Parameters:** An integer n, representing the position in the sequence (0-indexed).

?

- **Logic:**
  - The sequence starts: 2, 1, 3, 4, 7, 11, 18, 29, 47...
  - $L(0) = 2$
  - $L(1) = 1$
  - For $n > 1$, the formula is standard: $L(n) = L(n-1) + L(n-2)$
- **Implementation Note:** Use a loop (iteration) to calculate the value.
- **Return Value:** The Lucas number at index n.

3. fillCounterClockwiseSpiral

This is the core logic of the assignment. You must fill the matrix with consecutive Lucas numbers moving in a counter-clockwise spiral.

- **Parameters:** An integer n, the size of the matrix.
- **Algorithmic Guide:**
  Use the **"Boundary Shrinking"** technique again, but the order of movement is reversed compared to Lab A.
  Variables to Track:

  1. **Boundaries:** Define top, bottom, left, and right.
     - Initialize top to 0, bottom to $n-1$, left to 0, right to $n-1$.
  2. **Sequence Tracker:** Maintain a counter (e.g., lucasIndex) starting at 0.

     The Loop: Run a while loop that continues as long as top <= bottom AND left <= right. Inside this loop, perform the following movements:

  1. **Move DOWN (Left Column):**
     - Iterate a row index from top to bottom.
     - Set matrix[row][left] to getLucas(lucasIndex).
     - **Increment** lucasIndex.
     - After the loop, increment left (shift the left boundary right).
  2. **Move RIGHT (Bottom Row):**
     - Iterate a column index from left to right.
     - Set matrix[bottom][col] to getLucas(lucasIndex).

        - **Increment** lucasIndex.
        - After the loop, decrement bottom (shift the bottom boundary up).
    3. **Move UP (Right Column):**
        - *Check:* Verify if left <= right.
        - Iterate a row index from bottom down to top.
        - Set matrix[row][right] to getLucas(lucasIndex).
        - **Increment** lucasIndex.
        - After the loop, decrement right (shift the right boundary left).
    4. **Move LEFT (Top Row):**
        - *Check:* Verify if top <= bottom.
        - Iterate a column index from right down to left.
        - Set matrix[top][col] to getLucas(lucasIndex).
        - **Increment** lucasIndex.
        - After the loop, increment top (shift the top boundary down).
- **Return Value:** The fully populated 2D array.

4. calculateDiagonalSum

This method calculates the sum of elements residing on the two primary diagonals.

- **Parameters:** The 2D array (matrix).
- **Logic:**
    - **Main Diagonal:** Top-left to bottom-right.
    - **Anti-Diagonal:** Top-right to bottom-left.
    - **Intersection Rule:** If N is **odd**, the center element is shared. Ensure it is subtracted once from the total so it isn't double-counted.
- **Return Value:** The total integer sum.

Example Scenario

Input:

Enter matrix size N (max 6): 3

Logical Execution:

1. **Lucas Sequence:** 2, 1, 3, 4, 7, 11, 18, 29, 47...

2. **Spiral Filling (3×3) - Counter-Clockwise:**

   - **Down:** Left column fills with 2, 1, 3. Left boundary moves right.
   - **Right:** Bottom row fills with 4, 7. Bottom boundary moves up.
   - **Up:** Right column fills with 11, 18. Right boundary moves left.
   - **Left:** Top row fills with 29. Top boundary moves down.
   - **Down (Inner):** Center fills with 47.

Resulting Matrix:
2	29	18
1	**47**	**11**
3	**4**	**7**
Diagonal Calculation:

- Main Diagonal: $2 + 47 + 7 = 56$
- Anti-Diagonal: $18 + 47 + 3 = 68$
- *Correction (Center 47 is shared):* $56 + 68 - 47 = 77$

Output:

Diagonal Sum (Main + Anti): 77

Instructions for Submission

- Ensure your code compiles without errors.
- Do not modify the main method provided in the template.
- Follow the method signatures exactly as described above.

Requested files

SilverSpiral.java

```java
import java.util.Scanner;

/**
 * CS1 Lab B: The Silver Spiral
 * Fill a 2D matrix with Lucas numbers in a counter-clockwise spiral pattern.
 *
 * @author [Student Name]
 * @version 1.0
 */
public class SilverSpiral {

    /**
     * Main method - PROVIDED TO STUDENTS (DO NOT MODIFY)
     */
    public static void main(String[] args) {
        System.out.println("=== Lab B: The Silver Spiral ===");
        System.out.println("Fill a matrix with Lucas numbers in counter-clockwise spiral.");

        int n = getMatrixSize();

        int[][] spiral = fillCounterClockwiseSpiral(n);

        int diagonalSum = calculateDiagonalSum(spiral);
        System.out.println("\nDiagonal Sum (Main + Anti): " + diagonalSum);
    }

    // ========================================================================
    // STUDENT IMPLEMENTATION SECTION - COMPLETE THE METHODS BELOW
    // ========================================================================


    public static int getMatrixSize() {

    }

    public static int getLucas(int n) {

    }

    public static int[][] fillCounterClockwiseSpiral(int n) {

    }

    public static int calculateDiagonalSum(int[][] matrix) {

    }
}
```

VPL